CASE STUDY 1 – UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement:

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. Objectives

- Check minimum marks in every subject.
- Calculate aggregate percentage and validate attendance and family income.
- Separate eligible and ineligible applicants.
- Rank eligible applicants using aggregate, Mathematics marks, and alphabetical order.

3. Assumptions

- Three subjects are considered: Mathematics, Physics, and Computer Science; each is out of 100.
- Minimum subject mark is 50; minimum aggregate is 70%.
- Attendance must be at least 75%; annual family income must not exceed ₹2,50,000.
- Ranking: higher aggregate first, then higher Mathematics marks, then alphabetical name.

4. Algorithm

- Store applicants as tuples in a list.
- For each applicant, calculate aggregate and check every eligibility condition.
- Append eligible records with aggregate to one list; append rejected records and reasons to another list.
- Use nested loops to rank eligible records according to the tie-breaking rules.
- Display the ranked list and rejected applicants.

5. Pseudocode

START

Set eligibility thresholds.

Store applicant records as tuples.

FOR each applicant: calculate aggregate and check marks, attendance, and income.

IF all conditions pass, add applicant to eligible list; ELSE add to rejected list.

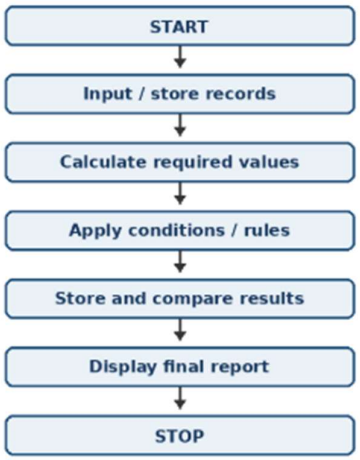
Compare eligible pairs using nested loops and swap when ranking order is wrong.

Display ranking and rejection reasons.

STOP

6. Flowchart

Process flow (high-level):



7. Python Source Code

```
minimum_subject_mark = 50
minimum_aggregate = 70
minimum_attendance = 75
maximum_income = 250000
applicants = [
```

```
    ("Anil", 85, 78, 90, 92, 180000),
    ("Bhavya", 75, 72, 80, 80, 220000),
    ("Charan", 75, 72, 80, 88, 200000),
    ("Divya", 65, 70, 72, 90, 150000),
    ("Esha", 90, 88, 92, 70, 120000)
]
eligible = []
rejected = []
for applicant in applicants:
    name, maths, physics, computer, attendance, income = applicant

    aggregate = (maths + physics + computer) / 3
    reason = ""

    if (maths < minimum_subject_mark or
        physics < minimum_subject_mark or
        computer < minimum_subject_mark):
        reason = "Subject mark below minimum"

    elif aggregate < minimum_aggregate:
        reason = "Aggregate below 70%"

    elif attendance < minimum_attendance:
        reason = "Attendance below 75%"

    elif income > maximum_income:
```

```
        reason = "Income above limit"

    if reason == "":
        eligible.append(
            (name, maths, physics, computer, attendance, income, aggregate)
        )
    else:
        rejected.append((name, round(aggregate, 2), reason))

for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        first = eligible[i]
        second = eligible[j]
        swap = False

        if first[6] < second[6]:
            swap = True

        elif first[6] == second[6] and first[1] < second[1]:
            swap = True

        elif (first[6] == second[6] and
              first[1] == second[1] and
              first[0] > second[0]):
            swap = True
        if swap:
```

```
            eligible[i], eligible[j] = eligible[j], eligible[i]

print("UNIVERSITY SCHOLARSHIP ALLOCATION")
print("--------------------------------")

rank = 1

for student in eligible:
    name, maths, physics, computer, attendance, income, aggregate = student
    print(
        "Rank:", rank,
        "| Name:", name,
        "| Aggregate:", round(aggregate, 2), "%"
    )
    rank += 1
print("\nINELIGIBLE APPLICANTS")
for student in rejected:
    name, aggregate, reason = student
    print(name, "| Aggregate:", aggregate, "% | Reason:", reason)
```

8. Test Cases

Test Case	Applicant	Maths	Physics	Computer Science	Attendance (%)	Annual Income (₹)
1	Anil	85	78	90	92	180000
2	Bhavya	75	72	80	80	220000
3	Charan	75	72	80	88	200000
4	Divya	65	70	72	90	150000
5	Esha	90	88	92	70	120000

9. Sample Output Screenshot:

```
UNIVERSITY SCHOLARSHIP ALLOCATION
--------------------------------
Rank: 1 | Name: Anil | Aggregate: 84.33 %
Rank: 2 | Name: Bhavya | Aggregate: 75.67 %
Rank: 3 | Name: Charan | Aggregate: 75.67 %

INELIGIBLE APPLICANTS
Divya | Aggregate: 69.0 % | Reason: Aggregate below 70%
Esha | Aggregate: 90.0 % | Reason: Attendance below 75%
```

10. Conclusion

The program applies academic, attendance, and income rules, then ranks qualified applicants with deterministic tie-breaking. It demonstrates variables, operators, conditions, loops, lists, tuples, and tuple unpacking.